HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN

BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN: IOT VÀ ỨNG DỤNG
MÃ HỌC PHẦN: INT14149

ĐỀ TÀI

HỆ THỐNG GIÁM SÁT VÀ ĐIỀU KHIỂN NHÀ KÍNH THÔNG MINH (SMART GREENHOUSE)

Giảng viên hướng dẫn:	**TS. Nguyễn Quốc Uy**
Sinh viên thực hiện:	**Đặng Quân Trường**
Mã sinh viên:	**B23DCAT306**
Chuyên ngành:	**An toàn thông tin**

HỌC KỲ 1 NĂM HỌC 2026-2027

LỜI CẢM ƠN

Để hoàn thành bài báo cáo bài tập lớn học phần "IoT và Ứng dụng", em xin chân thành cảm ơn thầy TS. Nguyễn Quốc Uy đã tận tình giảng dạy, định hướng và góp ý trong suốt quá trình học tập và thực hiện đề tài. Những kiến thức về IoT, giao thức MQTT và xây dựng hệ thống mà thầy truyền đạt là nền tảng giúp em hoàn thiện từ bản đặc tả SRS ở Báo cáo 1 đến một hệ thống chạy được từ phần cứng đến giao diện web.

Em xin chân thành cảm ơn!

Hà Nội, tháng 10 năm 2026
Sinh viên thực hiện
Đặng Quân Trường

MỤC LỤC

LỜI MỞ ĐẦU 7
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI 8
1.1 Giới thiệu đề tài 8
1.2 Mục tiêu đề tài 8
1.3 Phạm vi đề tài 8
1.4 Công nghệ sử dụng 9
CHƯƠNG 2. KIẾN TRÚC VÀ PHÂN TÍCH HỆ THỐNG 10
2.1 Kiến trúc tổng thể 10
2.2 Giao thức MQTT và các topic 11
2.3 Use case hệ thống 12
2.3.1 Sơ đồ use case 12
2.3.2 Đặc tả use case và biểu đồ tuần tự 13
CHƯƠNG 3. THIẾT KẾ VÀ HIỆN THỰC HỆ THỐNG 21
3.1 Thiết kế phần cứng và firmware 21
3.1.1 Linh kiện và sơ đồ kết nối 21
3.1.2 Nguyên lý hoạt động của mạch 21
3.1.3 Lập trình firmware 22
3.2 Thiết kế cơ sở dữ liệu 23
3.3 Thiết kế Backend 25
3.3.1 Cấu trúc mã nguồn 25
3.3.2 Các chức năng xử lý chính 25
3.4 Danh mục API 26
a) API đăng nhập, đăng xuất 26
b) API điều khiển thiết bị 27
c) API dữ liệu cảm biến và lịch sử 27
d) API dashboard 27
3.5 Thiết kế giao diện Frontend 27
3.5.1 Cơ chế hoạt động chung 27
3.5.2 Trang Dashboard 28
3.5.3 Trang Data Sensor 28
3.5.4 Trang Action History 29
3.5.5 Trang Profile 29
CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM 31
4.1 Mô hình phần cứng thực tế 31
4.2 Số liệu vận hành thực tế 31

4.3 Kịch bản kiểm thử 32
4.4 Đối chiếu kết quả với mục tiêu.....34
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 35
5.1 Các kết quả đã đạt được.....35
5.2 Những điểm hạn chế.....35
5.3 Hướng phát triển.....36
TÀI LIỆU THAM KHẢO 37

DANH MỤC CÁC HÌNH VẼ

Hình 1. Kiến trúc tổng thể của hệ thống Smart Greenhouse 10
Hình 2. Sơ đồ use case của hệ thống 12
Hình 3. Biểu đồ tuần tự UC01 - Đăng nhập và xác thực JWT 16
Hình 4. Biểu đồ tuần tự UC02, UC04 - Luồng dữ liệu cảm biến 17
Hình 5. Biểu đồ tuần tự UC03 - Điều khiển LED 19
Hình 6. Biểu đồ tuần tự UC05 - Tra cứu lịch sử thao tác 20
Hình 7. Sơ đồ nối chân giữa ESP8266 và các linh kiện 21
Hình 8. Sơ đồ cơ sở dữ liệu của hệ thống 23
Hình 9. Giao diện trang Dashboard 28
Hình 10. Giao diện trang Data Sensor 29
Hình 11. Giao diện trang Action History 29
Hình 12. Giao diện trang Profile 30
Hình 13. Mô hình phần cứng thực tế lắp trên breadboard 31
Hình 14. Kết quả lọc lịch sử theo kết quả FAILED 33

DANH MỤC CÁC BẢNG BIỂU

Bảng 1. Công nghệ, thư viện và công cụ sử dụng trong đề tài.....9
Bảng 2. Các topic MQTT của hệ thống.....11
Bảng 3. Danh sách use case.....13
Bảng 4. Đặc tả use case UC01 - Đăng nhập.....14
Bảng 5. Đặc tả use case UC02 - Giám sát cảm biến trên Dashboard.....16
Bảng 6. Đặc tả use case UC03 - Điều khiển LED.....18
Bảng 7. Đặc tả use case UC05 - Tra cứu lịch sử thao tác 20
Bảng 8. Linh kiện và chân kết nối với ESP8266 NodeMCU.....21
Bảng 9. Cấu trúc bảng sensor_data (dữ liệu cảm biến).....24
Bảng 10. Cấu trúc bảng action_history (lịch sử điều khiển).....24
Bảng 11. Danh mục API của hệ thống 26
Bảng 12. Thống kê dữ liệu trong cơ sở dữ liệu 32
Bảng 13. Thống kê lệnh điều khiển theo từng đèn.....32
Bảng 14. Các kịch bản kiểm thử và kết quả 33
Bảng 15. Đối chiếu kết quả thực nghiệm với mục tiêu đề ra 34

LỜI MỞ ĐẦU

Internet vạn vật (IoT - Internet of Things) cho phép các thiết bị vật lý thu thập dữ liệu từ môi trường, kết nối mạng và trao đổi thông tin với phần mềm để giám sát, điều khiển từ xa. Trong nông nghiệp công nghệ cao, mô hình nhà kính (Greenhouse) là một bài toán tiêu biểu: người trồng cần theo dõi liên tục nhiệt độ, độ ẩm, ánh sáng và có thể bật, tắt các thiết bị hỗ trợ mà không cần có mặt trực tiếp tại vườn.

Ở Báo cáo 1, đề tài "Smart Greenhouse Dashboard" được đặc tả ở mức tài liệu SRS: mô tả hệ thống, thiết kế cơ sở dữ liệu, use case và biểu đồ tuần tự. Ở báo cáo này, hệ thống đã được hiện thực thành một sản phẩm chạy được từ phần cứng đến giao diện: mạch ESP8266 NodeMCU đọc cảm biến DHT11 và quang trở, điều khiển 3 đèn LED; dữ liệu và lệnh điều khiển được truyền qua giao thức MQTT; Backend Node.js (Express) lưu dữ liệu vào SQLite và cung cấp REST API bảo vệ bằng JWT; giao diện web HTML/CSS/JavaScript hiển thị số liệu, biểu đồ, bảng tra cứu và cho phép bật/tắt đèn ngay trên trình duyệt.

Báo cáo gồm 5 chương:

- **Chương 1 - Giới thiệu đề tài:** mục tiêu, phạm vi và công nghệ sử dụng.
- **Chương 2 - Kiến trúc và phân tích hệ thống:** kiến trúc tổng thể, giao thức MQTT, use case và biểu đồ tuần tự.
- **Chương 3 - Thiết kế và hiện thực:** phần cứng và firmware, cơ sở dữ liệu, Backend, danh mục API và giao diện.
- **Chương 4 - Kết quả thực nghiệm:** mô hình thực tế, số liệu vận hành, kịch bản kiểm thử, đối chiếu mục tiêu.
- **Chương 5 - Kết luận và hướng phát triển.**

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Giới thiệu đề tài

Đề tài **"Xây dựng hệ thống giám sát và điều khiển nhà kính thông minh (Smart Greenhouse)"** thu thập ba thông số môi trường là nhiệt độ, độ ẩm không khí và cường độ ánh sáng, truyền về máy chủ theo thời gian thực, lưu trữ để tra cứu và cho phép người dùng điều khiển 3 thiết bị chấp hành (mô phỏng bằng 3 đèn LED) từ xa qua giao diện web.

So với bản đặc tả ở Báo cáo 1, quá trình hiện thực đã đơn giản hóa và cụ thể hóa một số lựa chọn công nghệ để hệ thống chạy được trọn vẹn trên máy tính cá nhân:

- Vi điều khiển: **ESP8266 NodeMCU** (thay cho phương án "ESP32 hoặc ESP8266" dự kiến).
- Giao diện: **HTML, CSS, JavaScript thuần** (thay cho React.js / Next.js dự kiến).
- Cơ sở dữ liệu: **SQLite** với 2 bảng `sensor_data` và `action_history` (thay cho MySQL/PostgreSQL 5 bảng dự kiến).
- Thiết bị điều khiển: **3 đèn LED** (LED01, LED02, LED03) thay cho đèn quang hợp, quạt và máy bơm; cảm biến gồm DHT11 và quang trở (LDR).

1.2 Mục tiêu đề tài

- **Thu thập dữ liệu môi trường:** ESP8266 đọc nhiệt độ, độ ẩm (DHT11) và ánh sáng (quang trở) mỗi 3 giây, gửi lên MQTT dưới dạng JSON.
- **Điều khiển có xác nhận:** bật/tắt từng đèn hoặc tất cả đèn từ web; kết quả (SUCCESS / FAILED) chỉ được ghi nhận dựa trên phản hồi thật của thiết bị, có thời gian chờ tối đa 5 giây.
- **Lưu trữ:** lưu toàn bộ dữ liệu cảm biến và lịch sử điều khiển vào cơ sở dữ liệu SQLite.
- **Backend và API:** cung cấp REST API xác thực bằng JWT để đăng nhập, đăng xuất, lấy dữ liệu cảm biến, lịch sử và gửi lệnh điều khiển.
- **Giao diện web:** form đăng nhập và 4 trang Dashboard, Data Sensor, Action History, Profile; có biểu đồ, tìm kiếm, lọc, sắp xếp, phân trang và tự cập nhật dữ liệu.

1.3 Phạm vi đề tài

- **Phần cứng:** ESP8266 NodeMCU, cảm biến DHT11, quang trở LDR (đọc qua chân analog A0), 3 đèn LED, breadboard và dây nối.

- **Truyền thông:** giao thức MQTT, 3 topic `data_sensors`, `device_control`, `device_response`; broker chạy cục bộ, có xác thực user/password.
- **Phần mềm máy chủ:** Backend Node.js (Express) chạy ở cổng 3000, đồng thời phục vụ các tệp giao diện web.
- **Cơ sở dữ liệu:** SQLite (tệp `greenhouse.db`).
- **Ngoài phạm vi:** mã hóa đường truyền (TLS/HTTPS), quản lý nhiều người dùng, ứng dụng di động, điều khiển tự động theo ngưỡng cảm biến.

1.4 Công nghệ sử dụng

Bảng 1. Công nghệ, thư viện và công cụ sử dụng trong đề tài

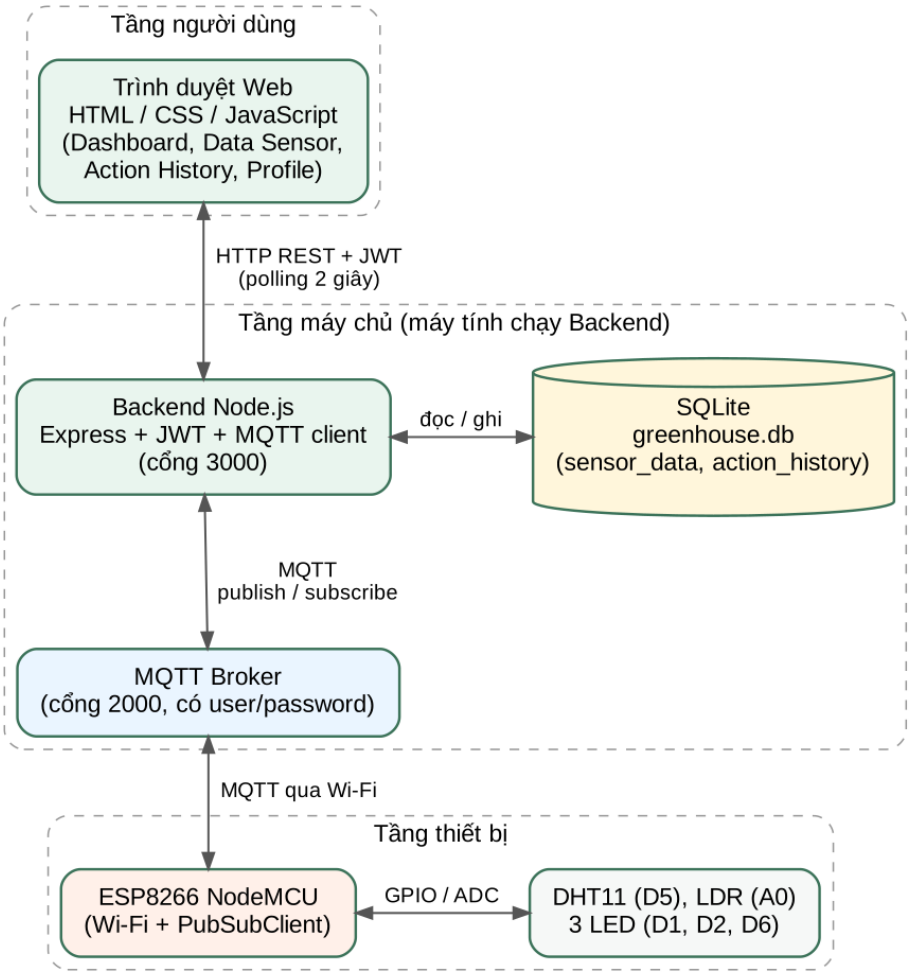
Thành phần	Công nghệ	Vai trò
Vi điều khiển	ESP8266 NodeMCU	Kết nối Wi-Fi và MQTT, đọc cảm biến, điều khiển 3 chân LED
Cảm biến	DHT11; quang trở LDR	Đo nhiệt độ, độ ẩm không khí; đo cường độ ánh sáng (giá trị ADC 0-1023)
Cơ cấu chấp hành	3 đèn LED	Mô phỏng 3 thiết bị bật/tắt trong nhà kính
Lập trình firmware	C/C++ (Arduino framework, tệp .ino); thư viện ESP8266WiFi, PubSubClient, DHT	Đọc cảm biến, nhận lệnh, gửi dữ liệu và phản hồi qua MQTT
MQTT Broker	Broker MQTT chạy trên máy tính (cổng 2000)	Trung gian định tuyến bản tin giữa ESP8266 và Backend
Backend	Node.js; express, mqtt, jsonwebtoken, cors, sqlite3	REST API, xác thực JWT, cầu nối MQTT - CSDL, phục vụ giao diện
Cơ sở dữ liệu	SQLite	Lưu dữ liệu cảm biến và lịch sử điều khiển
Giao diện	HTML5, CSS3, JavaScript thuần, Fetch API, Canvas API	Dashboard, biểu đồ, bảng tra cứu, điều khiển thiết bị
Thiết kế giao diện	Figma	Phác thảo bố cục các màn hình (Báo cáo 1)

CHƯƠNG 2. KIẾN TRÚC VÀ PHÂN TÍCH HỆ THỐNG

2.1 Kiến trúc tổng thể

Hệ thống được tổ chức thành ba tầng độc lập, giao tiếp với nhau qua hai giao thức: HTTP (giữa trình duyệt và Backend) và MQTT (giữa Backend và thiết bị).

- **Tầng thiết bị:** ESP8266 NodeMCU nối với DHT11, quang trở và 3 LED. Thiết bị đọc cảm biến, publish dữ liệu lên MQTT Broker, subscribe nhận lệnh điều khiển và publish phản hồi sau khi thực hiện lệnh.
- **Tầng máy chủ:** MQTT Broker định tuyến bản tin; Backend Node.js vừa là MQTT client (subscribe dữ liệu và phản hồi, publish lệnh), vừa là máy chủ REST API; SQLite lưu dữ liệu.
- **Tầng người dùng:** trình duyệt chạy giao diện web, gọi REST API kèm JWT và tự làm mới dữ liệu mỗi 2 giây.



Hình 1. Kiến trúc tổng thể của hệ thống Smart Greenhouse

Hệ thống có hai luồng dữ liệu chính:

- **Luồng đi lên (giám sát):** ESP8266 → MQTT Broker → Backend → SQLite → (REST API) → trình duyệt.

- **Luồng đi xuống (điều khiển):** trình duyệt → (REST API) → Backend → MQTT Broker → ESP8266; sau khi thực hiện, ESP8266 gửi phản hồi theo chiều ngược lại để Backend ghi lịch sử và trả kết quả cho giao diện.

2.2 Giao thức MQTT và các topic

MQTT là giao thức truyền thông nhẹ theo mô hình publish/subscribe, phù hợp với vi điều khiển có tài nguyên hạn chế. Cả ESP8266 và Backend đều kết nối tới cùng một broker bằng tài khoản có xác thực (username/password); mỗi bên có Client ID riêng (`ESP8266-01` cho thiết bị, `NodeJS_Backend_<ngẫu nhiên>` cho Backend). Hệ thống dùng 3 topic, thể hiện trong Bảng 2.

Bảng 2. Các topic MQTT của hệ thống

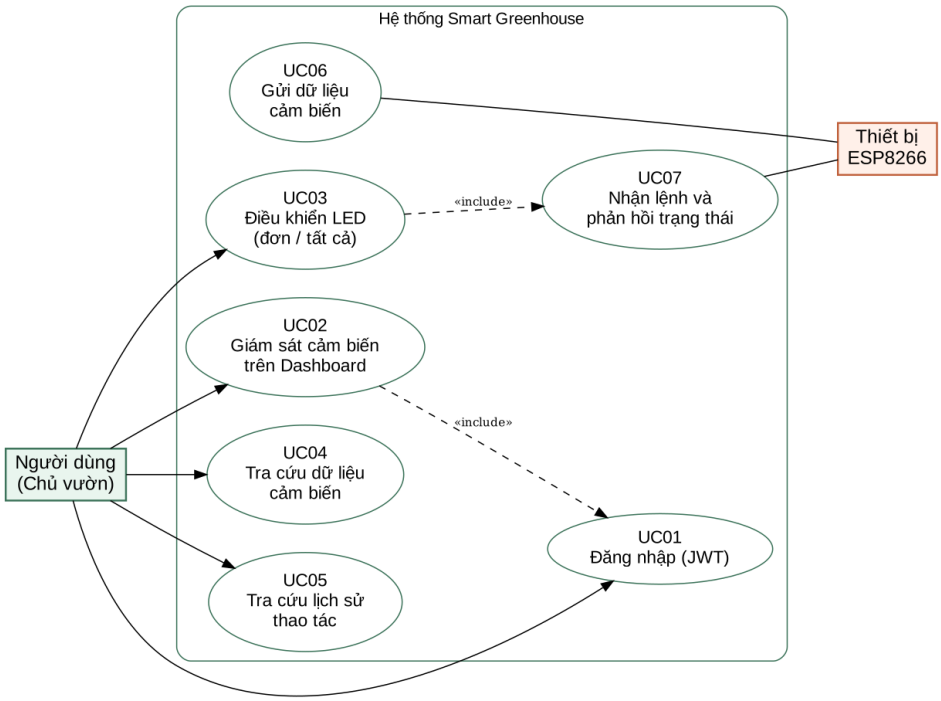
Topic	Chiều truyền	Payload mẫu (JSON)
`data_sensors`	ESP8266 → Backend	`{"device_id":"ESP8266-01","temperature":26.5,"humidity":56.5,"light":723}`
`device_control`	Backend → ESP8266	`{"device_id":"ESP8266-01","device":"LED01","action":"`
`device_response`	ESP8266 → Backend	`{"device_id":"ESP8266-01","device":"LED01","action":"ON","state":"ON","sta`

Việc tách riêng topic lệnh và topic phản hồi giúp Backend biết chắc thiết bị đã thực sự thực hiện lệnh hay chưa, thay vì chỉ giả định lệnh đã đến nơi. Các bản tin sử dụng mức chất lượng dịch vụ mặc định QoS 0 (gửi tối đa một lần).

2.3 Use case hệ thống

2.3.1 Sơ đồ use case

Hệ thống có hai tác nhân: **Người dùng** (chủ vườn) tương tác qua giao diện web; **Thiết bị ESP8266** tự động gửi dữ liệu và thực hiện lệnh. So với Báo cáo 1, use case "tự động điều khiển theo cảm biến ánh sáng" chưa được hiện thực nên không có trong phiên bản này (xem mục 5.2).



Hình 2. Sơ đồ use case của hệ thống

Bảng 3. Danh sách use case

Mã	Use case	Tác nhân	Mô tả
UC01	Đăng nhập	Người dùng	Nhập tài khoản trên form đăng nhập để lấy JWT gọi các API được bảo vệ; có thể đăng xuất
UC02	Giám sát cảm biến trên Dashboard	Người dùng	Xem nhiệt độ, độ ẩm, ánh sáng mới nhất và biểu đồ biến thiên
UC03	Điều khiển LED	Người dùng, ESP8266	Bật/tắt từng đèn hoặc tất cả đèn, chờ phản hồi từ thiết bị
UC04	Tra cứu dữ liệu cảm biến	Người dùng	Tìm kiếm, sắp xếp, phân trang dữ liệu cảm biến
UC05	Tra cứu lịch sử thao tác	Người dùng	Lọc theo thiết bị, hành động, kết quả, ngày; sắp xếp, phân trang
UC06	Gửi dữ liệu cảm biến	ESP8266	Định kỳ đọc cảm biến và publish dữ liệu lên MQTT
UC07	Nhận lệnh và phản hồi trạng thái	ESP8266	Nhận lệnh từ topic điều khiển, bật/tắt LED, gửi phản hồi

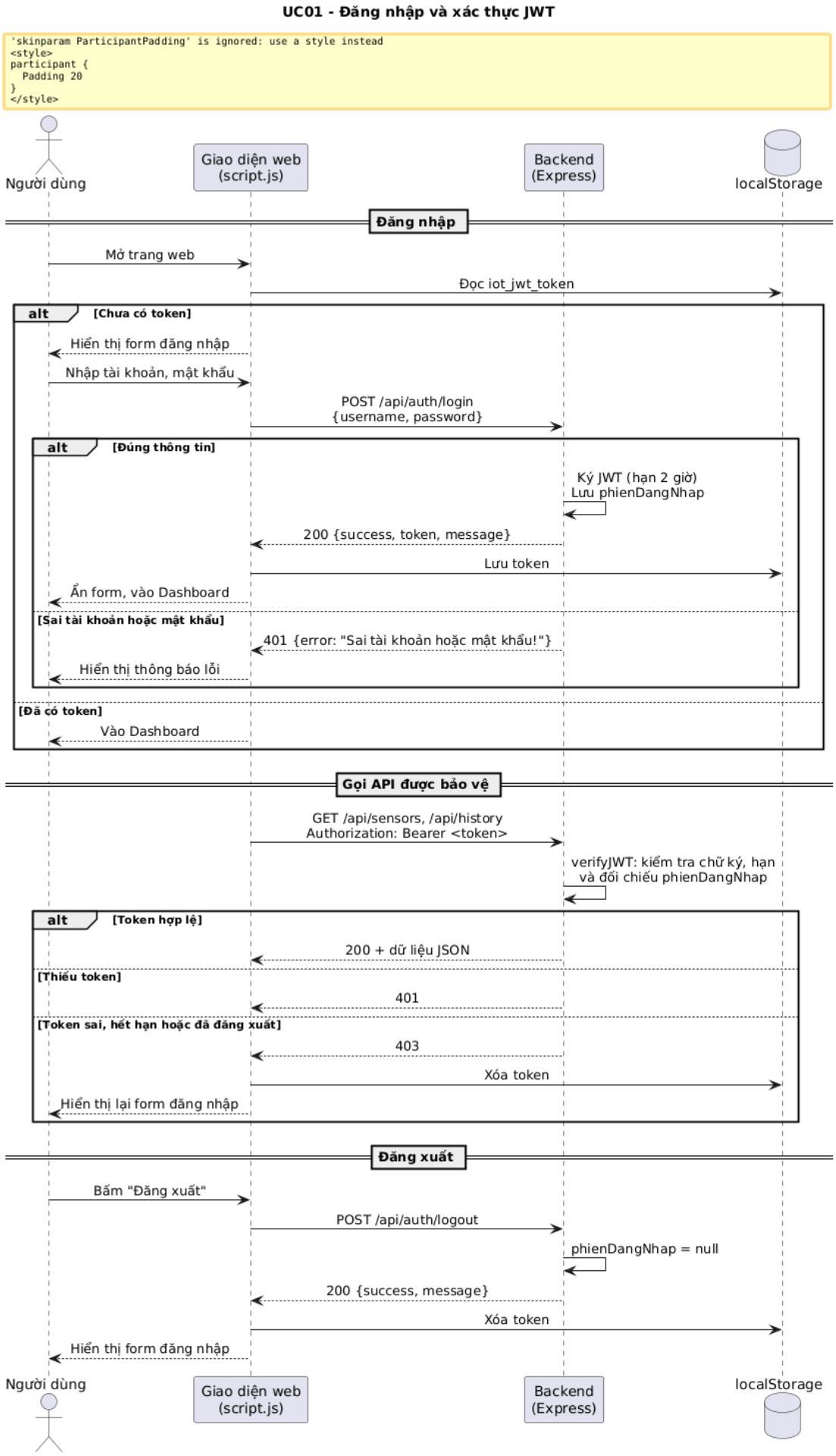
2.3.2 Đặc tả use case và biểu đồ tuần tự

UC01 - Đăng nhập. Backend cung cấp API đăng nhập trả về JWT có hạn 2 giờ. Giao diện hiển thị form đăng nhập (tài khoản, mật khẩu) khi chưa có token; sau khi đăng nhập thành công, tệp `script.js` lưu token vào `localStorage` và đính kèm vào mọi yêu

cầu sau đó. Nút "Đăng xuất" ở thanh bên trái hủy phiên đăng nhập và đưa người dùng về form đăng nhập.

Bảng 4. Đặc tả use case UC01 - Đăng nhập

Tác nhân	Người dùng (thông qua giao diện web)
Tiền điều kiện	Backend đang chạy ở cổng 3000; người dùng đã mở giao diện web.
Hậu điều kiện	Giao diện có JWT hợp lệ và gọi được các API được bảo vệ.
Luồng chính	1. Người dùng nhập tài khoản, mật khẩu vào form đăng nhập; giao diện gửi POST /api/auth/login kèm username và password. 2. Backend đối chiếu thông tin, ký JWT (hạn 2 giờ), lưu phiên đăng nhập và trả về token. 3. Giao diện lưu token vào localStorage, ẩn form đăng nhập và bắt đầu tải dữ liệu. 4. Mọi API sau đó gửi header Authorization: Bearer <token>; middleware verifyJWT kiểm tra chữ ký, hạn của token và đối chiếu với phiên đăng nhập hiện tại. 5. Khi người dùng bấm "Đăng xuất", giao diện gọi POST /api/auth/logout, xóa token khỏi localStorage và hiển thị lại form đăng nhập.
Luồng ngoại lệ	Sai tài khoản hoặc mật khẩu: Backend trả HTTP 401 và form đăng nhập hiển thị thông báo lỗi; nếu không kết nối được Backend, form báo "Không kết nối được Backend!". Thiếu token: HTTP 401. Token sai, hết hạn hoặc đã đăng xuất: HTTP 403; giao diện tự đăng xuất và hiển thị lại form đăng nhập.

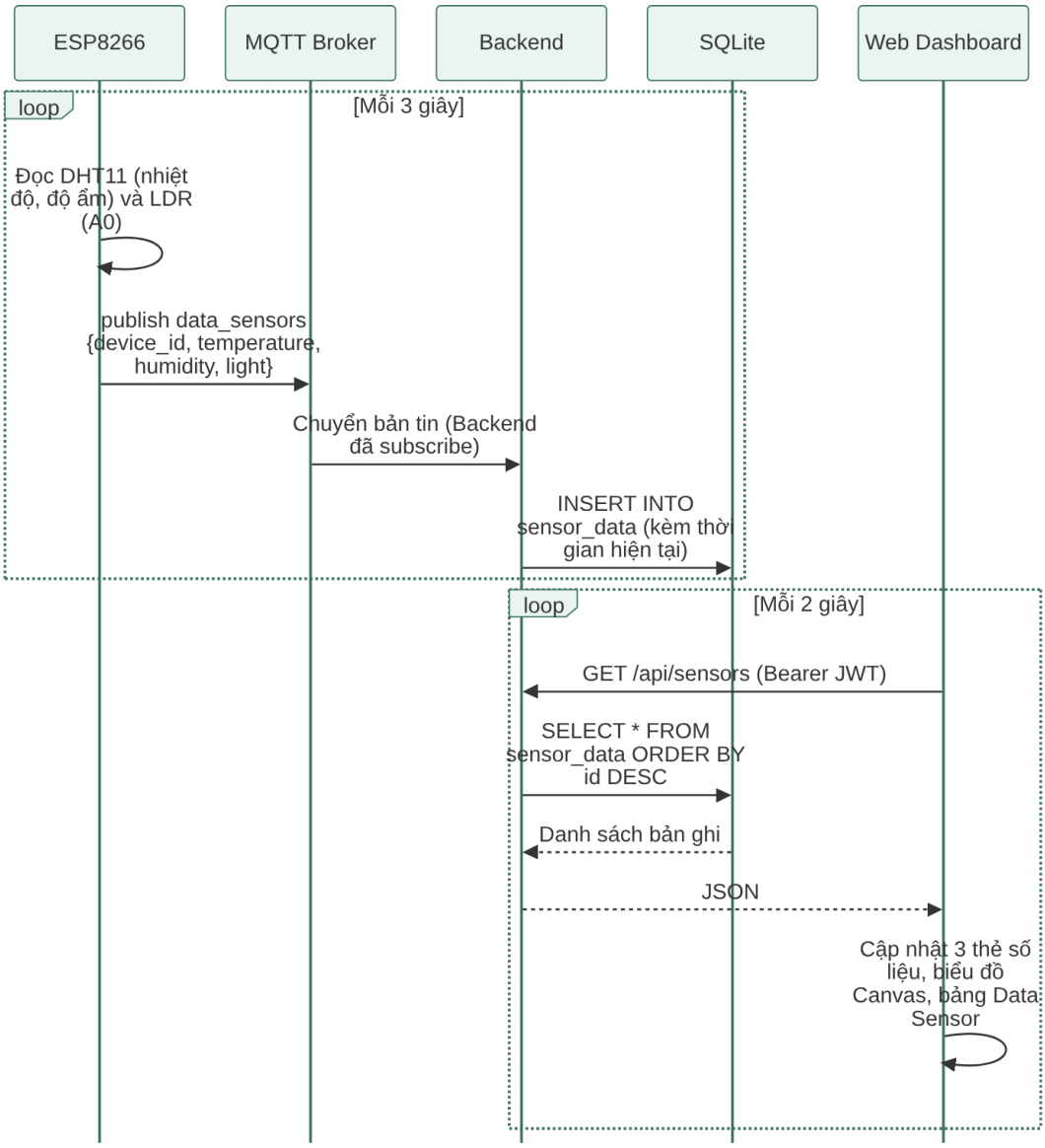

UC01 - Đăng nhập và xác thực JWT
'skinparam ParticipantPadding' is ignored: use a style instead
<style>
participant {
Padding 20
}
</style>
Người dùng
Giao diện web (script.js)
Backend (Express)
localStorage
Đăng nhập
Mở trang web
Đọc iot_jwt_token
alt [Chưa có token]
Hiển thị form đăng nhập
Nhập tài khoản, mật khẩu
POST /api/auth/login {username, password}
alt [Đúng thông tin]
Ký JWT (hạn 2 giờ) Lưu phienDangNhap
200 {success, token, message}
Lưu token
Ẩn form, vào Dashboard
[Sai tài khoản hoặc mật khẩu]
401 {error: "Sai tài khoản hoặc mật khẩu!"}
Hiển thị thông báo lỗi
[Đã có token]
Vào Dashboard
Gọi API được bảo vệ
GET /api/sensors, /api/history Authorization: Bearer <token>
verifyJWT: kiểm tra chữ ký, hạn và đối chiếu phienDangNhap
alt [Token hợp lệ]
200 + dữ liệu JSON
[Thiếu token]
401
[Token sai, hết hạn hoặc đã đăng xuất]
403
Xóa token
Hiển thị lại form đăng nhập
Đăng xuất
Bấm "Đăng xuất"
POST /api/auth/logout
phienDangNhap = null
200 {success, message}
Xóa token
Hiển thị form đăng nhập
Người dùng
Giao diện web (script.js)
Backend (Express)
localStorage

Hình 3. Biểu đồ tuần tự UC01 - Đăng nhập và xác thực JWT

UC02 và UC04 - Giám sát và tra cứu dữ liệu cảm biến. ESP8266 gửi dữ liệu mỗi 3 giây; Backend lưu vào SQLite; giao diện hỏi lại Backend mỗi 2 giây để cập nhật thẻ số liệu, biểu đồ và bảng Data Sensor.

Bảng 5. Đặc tả use case UC02 - Giám sát cảm biến trên Dashboard

Tác nhân	Người dùng; ESP8266 (phía thiết bị)
Tiền điều kiện	ESP8266 đã kết nối Wi-Fi và MQTT; người dùng đã có JWT.
Hậu điều kiện	Dashboard hiển thị giá trị mới nhất và biểu đồ các bản ghi gần nhất.
Luồng chính	1. ESP8266 đọc DHT11 và quang trở, publish JSON lên topic data_sensors. 2. Backend nhận bản tin, ghi thêm thời gian hiện tại và INSERT vào bảng sensor_data. 3. Giao diện gọi GET /api/sensors mỗi 2 giây. 4. Giao diện lấy bản ghi mới nhất cho 3 thẻ số liệu và vẽ biểu đồ Canvas theo số bản ghi người dùng chọn (7, 14, 30 hoặc 36).
Luồng ngoại lệ	DHT11 đọc lỗi (NaN): ESP8266 bỏ qua chu kỳ đó, không gửi bản tin. Chưa có dữ liệu: thẻ số liệu hiển thị "--" và biểu đồ để trống.

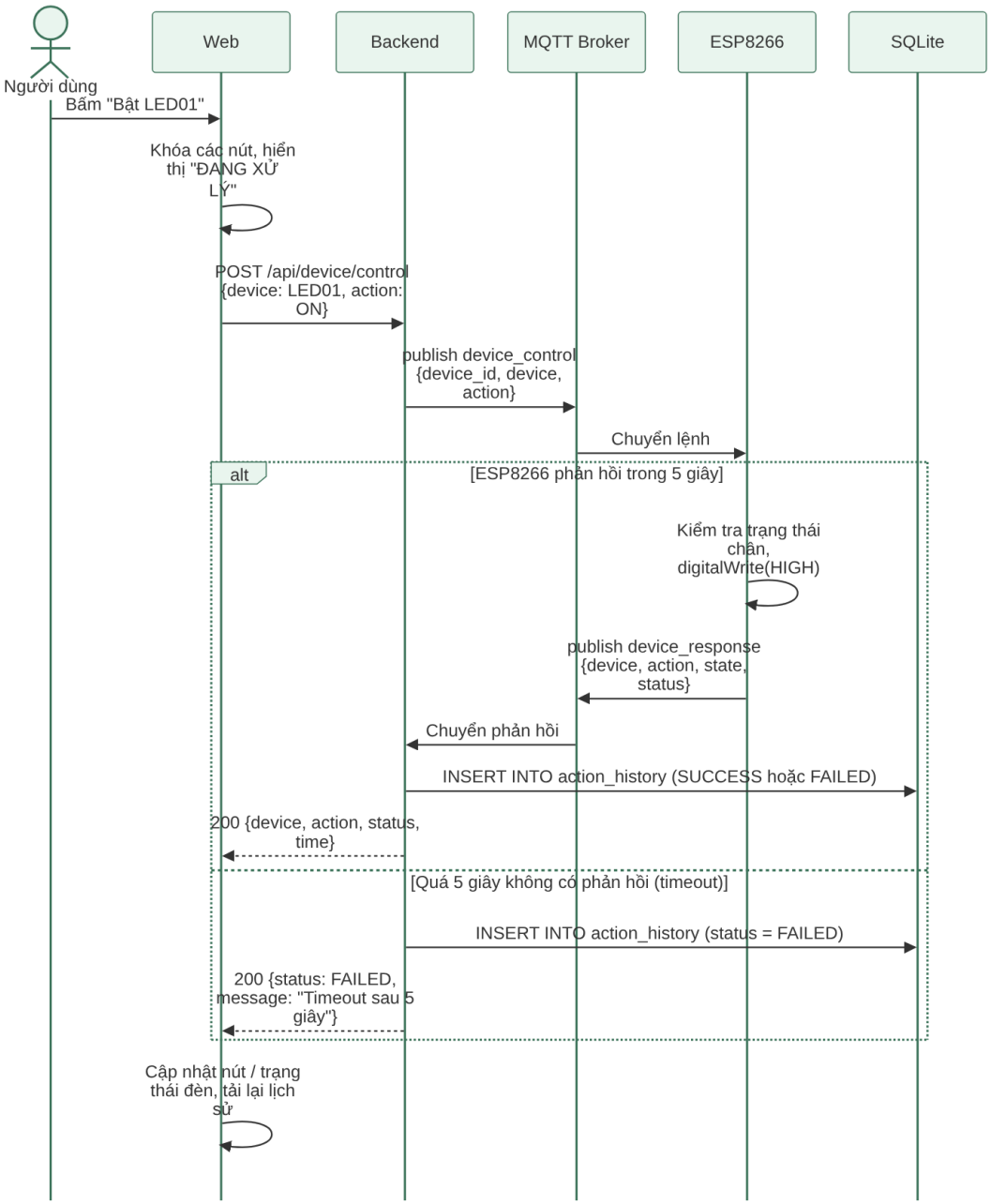


Hình 4. Biểu đồ tuần tự UC02, UC04 - Luồng dữ liệu cảm biến

UC03 - Điều khiển LED. Đây là use case quan trọng nhất vì đi qua toàn bộ chuỗi giao diện - Backend - MQTT - thiết bị và quay ngược lại.

Bảng 6. Đặc tả use case UC03 - Điều khiển LED

Tác nhân	Người dùng; ESP8266
Tiền điều kiện	Người dùng có JWT hợp lệ; ESP8266 đang kết nối MQTT.
Hậu điều kiện	LED đổi trạng thái; bản ghi kết quả được lưu trong action_history; giao diện cập nhật theo kết quả.
Luồng chính	1. Người dùng bấm nút bật/tắt một đèn hoặc "Bật tất cả" / "Tắt tất cả". 2. Giao diện khóa các nút, hiển thị trạng thái "ĐANG XỬ LÝ" và gửi POST /api/device/control {device, action}. 3. Backend publish lệnh lên topic device_control rồi chờ phản hồi tối đa 5 giây. 4. ESP8266 thực hiện lệnh và publish phản hồi lên topic device_response. 5. Backend ghi kết quả vào action_history (riêng ALL_LED được tách thành 3 dòng LED01, LED02, LED03) và trả kết quả cho giao diện. 6. Giao diện mở khóa nút, cập nhật trạng thái đèn và tải lại lịch sử.
Luồng ngoại lệ	Quá 5 giây không có phản hồi (thiết bị mất kết nối): Backend ghi FAILED và trả message báo lỗi timeout. Lệnh trùng trạng thái hiện tại (ví dụ bật đèn đang sáng): ESP8266 không đổi gì và phản hồi FAILED.

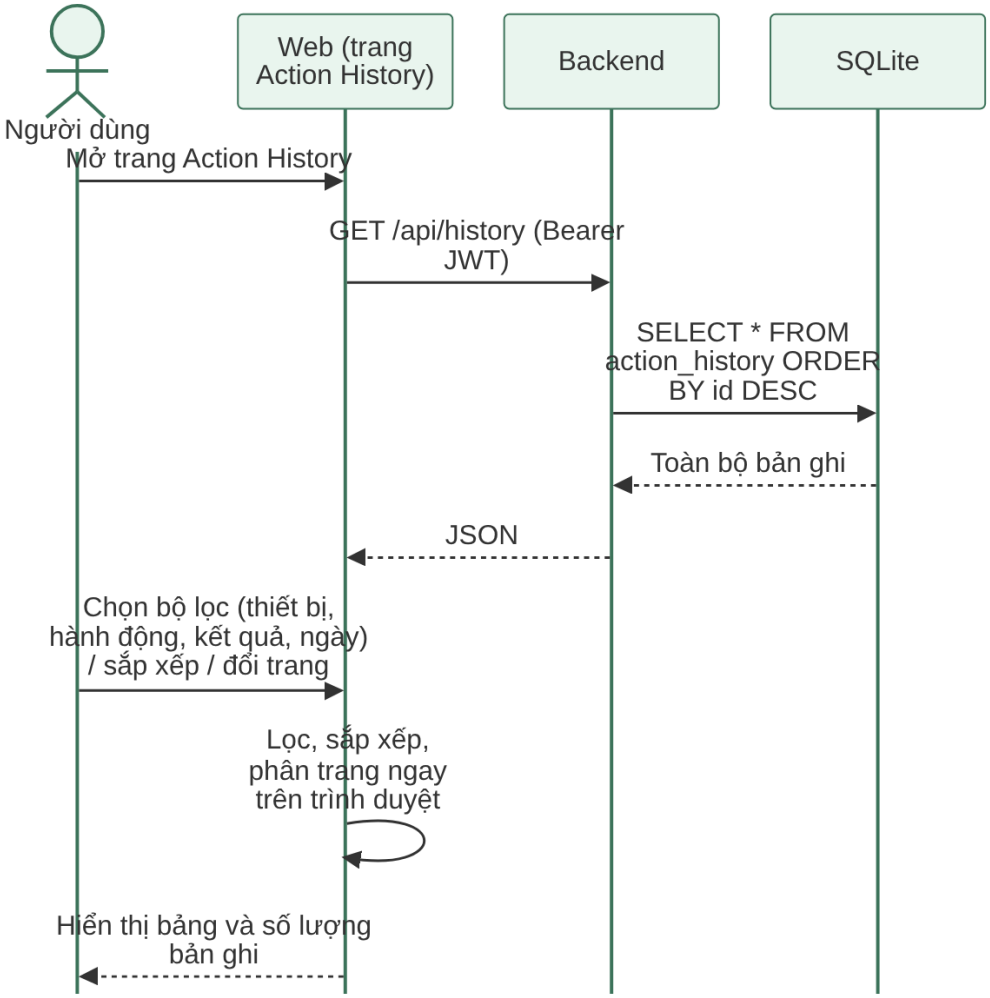


Hình 5. Biểu đồ tuần tự UC03 - Điều khiển LED

UC05 - Tra cứu lịch sử thao tác. Giao diện tải toàn bộ lịch sử từ Backend, sau đó lọc, sắp xếp và phân trang ngay trên trình duyệt.

Bảng 7. Đặc tả use case UC05 - Tra cứu lịch sử thao tác

Tác nhân	Người dùng
Tiền điều kiện	Người dùng có JWT hợp lệ.
Hậu điều kiện	Bảng lịch sử hiển thị đúng các bản ghi thỏa bộ lọc.
Luồng chính	1. Người dùng mở trang Action History; giao diện gọi GET /api/history. 2. Người dùng chọn bộ lọc: thiết bị (LED01, LED02, LED03), hành động (ON/OFF), kết quả (SUCCESS/FAILED), ngày (YYYY/MM/DD). 3. Giao diện lọc dữ liệu, cho phép sắp xếp theo từng cột và chọn 5, 10 hoặc 20 dòng mỗi trang.
Luồng ngoại lệ	Không có bản ghi phù hợp: bảng hiển thị "Không có lịch sử phù hợp".



Hình 6. Biểu đồ tuần tự UC05 - Tra cứu lịch sử thao tác

CHƯƠNG 3. THIẾT KẾ VÀ HIỆN THỰC HỆ THỐNG

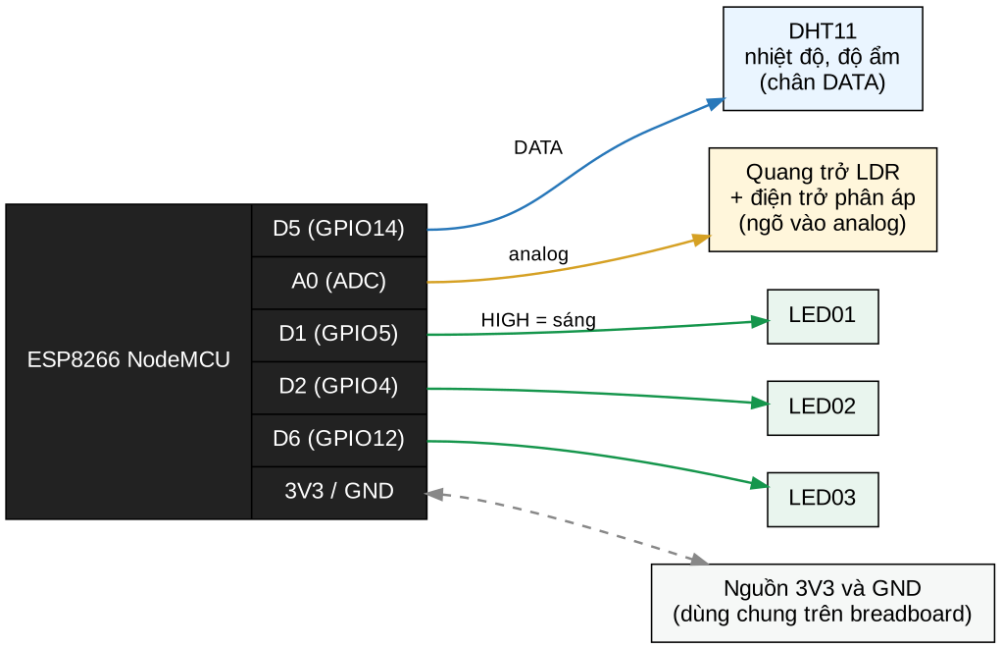
3.1 Thiết kế phần cứng và firmware

3.1.1 Linh kiện và sơ đồ kết nối

Các linh kiện được nối với ESP8266 NodeMCU theo các chân khai báo ở đầu tệp firmware `Bai2_IOT.ino` (Bảng 8).

Bảng 8. Linh kiện và chân kết nối với ESP8266 NodeMCU

Thành phần	Chức năng	Chân kết nối
ESP8266 NodeMCU	Vi điều khiển trung tâm, kết nối Wi-Fi và MQTT	-
DHT11	Đo nhiệt độ (°C) và độ ẩm không khí (%)	D5 (GPIO14) - chân DATA
Quang trở LDR	Đo cường độ ánh sáng, đọc dạng giá trị ADC	A0 (ngõ vào analog)
LED01	Thiết bị điều khiển số 1	D1 (GPIO5)
LED02	Thiết bị điều khiển số 2	D2 (GPIO4)
LED03	Thiết bị điều khiển số 3	D6 (GPIO12)
Breadboard, dây nối	Lắp ráp mạch, dùng chung nguồn 3V3 và GND	-



Hình 7. Sơ đồ nối chân giữa ESP8266 và các linh kiện

3.1.2 Nguyên lý hoạt động của mạch

- **LED:** mức HIGH trên chân GPIO thì đèn sáng, mức LOW thì đèn tắt. Khi khởi động, firmware đưa cả 3 chân về LOW nên cả 3 đèn tắt.

- **DHT11:** truyền dữ liệu nhiệt độ, độ ẩm qua một chân tín hiệu số duy nhất (D5), đọc bằng thư viện DHT.
- **Quang trở:** điện trở thay đổi theo ánh sáng, được đưa vào ngõ vào analog A0 và đọc bằng `analogRead` (0-1023). Firmware tính `light = 1023 - raw` để chỉ số tăng khi môi trường sáng hơn.

3.1.3 Lập trình firmware

Firmware viết bằng C/C++ trên Arduino framework, dùng các thư viện ESP8266WiFi (kết nối Wi-Fi), PubSubClient (MQTT client) và DHT (đọc cảm biến). Các hàm chính:

- **`setup()`:** cấu hình 3 chân LED là OUTPUT và tắt cả 3, khởi tạo DHT11, kết nối Wi-Fi, đặt địa chỉ MQTT broker và hàm callback, rồi kết nối MQTT.
- **`connectMQTT()`:** kết nối broker bằng Client ID `ESP8266-01` cùng username/password; thành công thì subscribe topic `device_control`; thất bại thì thử lại sau 2 giây.
- **`loop()`:** giữ kết nối MQTT, gọi `mqttClient.loop()` và cứ mỗi 3 giây (đếm bằng `millis()`, không dùng `delay()` nên không chặn việc nhận lệnh) thì gọi `publishSensorData()`.
- **`publishSensorData()`:** đọc DHT11 và quang trở, bỏ qua chu kỳ nếu DHT11 trả về NaN, đóng gói JSON và publish lên `data_sensors`.
- **`callback()`:** nhận bản tin trên `device_control`, xác định thiết bị (LED01, LED02, LED03 hoặc ALL_LED) và hành động (ON/OFF) rồi thực hiện.
- **`handleLEDCommand()` và `sendDeviceResponse()`:** đổi trạng thái chân LED và publish kết quả lên `device_response`.

Quy tắc phản hồi: nếu lệnh làm thay đổi trạng thái đèn thì phản hồi **SUCCESS**; nếu đèn đã ở đúng trạng thái được yêu cầu thì không làm gì và phản hồi **FAILED**. Với lệnh

ALL_LED, kết quả là FAILED khi cả 3 đèn đã ở trạng thái đó, ngược lại thiết bị đặt cả 3 đèn và phản hồi SUCCESS.

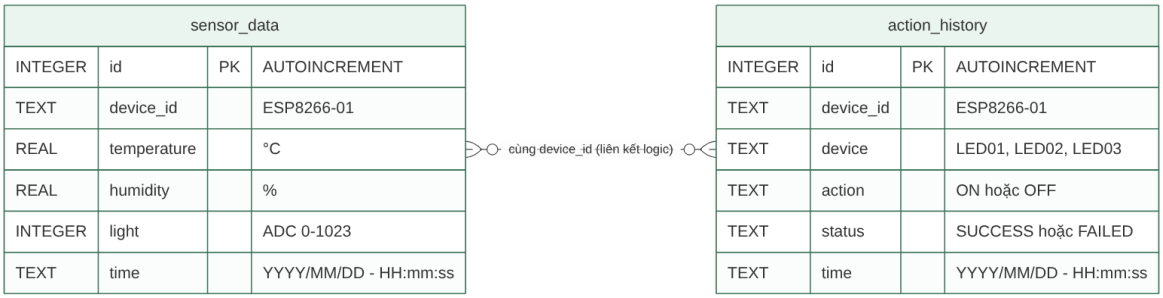
```
void handleLEDCommand(const char* device, const char* action, int ledPin) {
  int currentState = digitalRead(ledPin);
  if (strcmp(action, "ON") == 0) {
    if (currentState == LOW) {            // đèn đang tắt -> bật
      digitalWrite(ledPin, HIGH);
      sendDeviceResponse(device, "ON", "ON", "SUCCESS");
    } else {                              // đèn đã bật sẵn
      sendDeviceResponse(device, "ON", "ON", "FAILED");
    }
  } else if (strcmp(action, "OFF") == 0) {
    /* tương tự: HIGH -> LOW là SUCCESS, đã tắt sẵn là FAILED */
  }
}


void publishSensorData() {
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();
  if (isnan(humidity) || isnan(temperature)) return;  // DHT11 đọc lỗi
  int light = 1023 - analogRead(A0);
  snprintf(payload, sizeof(payload),
    "{\"device_id\":\"%s\",\"temperature\":%.1f,"
    "\"humidity\":%.1f,\"light\":%d}",
    DEVICE_ID, temperature, humidity, light);
  mqttClient.publish(TOPIC_SENSOR, payload);
}
```

Trích đoạn firmware (rút gọn): xử lý lệnh LED và gửi dữ liệu cảm biến

3.2 Thiết kế cơ sở dữ liệu

Hệ thống dùng SQLite, lưu trong tệp `greenhouse.db`. Khi Backend khởi động, tệp `database.js` tự tạo 2 bảng bằng câu lệnh `CREATE TABLE IF NOT EXISTS`. Bảng `sensor_data` lưu chuỗi dữ liệu đo, bảng `action_history` lưu lịch sử các lệnh điều khiển. Hai bảng không có khóa ngoại; chúng cùng gắn với một thiết bị thông qua trường `device_id` (giá trị `ESP8266-01`).



Hình 8. Sơ đồ cơ sở dữ liệu của hệ thống

Bảng 9. Cấu trúc bảng sensor_data (dữ liệu cảm biến)

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Mã bản ghi
device_id	TEXT	-	Mã thiết bị gửi dữ liệu (ESP8266-01)
temperature	REAL	-	Nhiệt độ (°C)
humidity	REAL	-	Độ ẩm không khí (%)
light	INTEGER	-	Cường độ ánh sáng (ADC, 0-1023)
time	TEXT	-	Thời điểm Backend nhận bản tin, định dạng YYYY/MM/DD - HH:mm:ss

Bảng 10. Cấu trúc bảng action_history (lịch sử điều khiển)

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Mã bản ghi
device_id	TEXT	-	Mã thiết bị phần cứng (ESP8266-01)
device	TEXT	-	Đèn bị tác động: LED01, LED02 hoặc LED03
action	TEXT	-	Hành động: ON hoặc OFF
status	TEXT	-	Kết quả: SUCCESS hoặc FAILED
time	TEXT	-	Thời điểm ghi nhận, định dạng YYYY/MM/DD - HH:mm:ss

Một số quyết định thiết kế:

- Lệnh **ALL_LED** được Backend tách thành 3 dòng LED01, LED02, LED03 khi lưu, nhờ vậy lịch sử và bộ lọc theo thiết bị luôn thống nhất; riêng phản hồi API vẫn giữ tên ALL_LED.
- Thời gian lưu dạng văn bản `YYYY/MM/DD - HH:mm:ss` theo giờ của máy chủ (không dùng giờ của ESP8266, vì thiết bị không có đồng hồ thực), giúp giao diện lọc theo ngày bằng cách so sánh chuỗi.
- Trạng thái hiện tại của mỗi đèn không lưu thành bảng riêng mà được suy ra từ bản ghi SUCCESS gần nhất trong `action_history`.

3.3 Thiết kế Backend

3.3.1 Cấu trúc mã nguồn

Mã nguồn gồm phần giao diện ở thư mục gốc và phần Backend trong thư mục iot-backend. Backend phục vụ trực tiếp các tệp giao diện bằng `express.static`, nên chỉ cần mở `http://localhost:3000` là dùng được cả hệ thống.

```
thư-mục-dự-án/
├── index.html        giao diện: form đăng nhập + 4 trang (Dashboard, ...)
├── style.css         định dạng giao diện
├── script.js         gọi API, vẽ biểu đồ, lọc, sắp xếp, phân trang
├── docs/             tài liệu API và báo cáo (tệp PDF)
└── iot-backend/
    ├── server.js     Express + MQTT client + REST API + JWT
    ├── database.js   khởi tạo SQLite, tạo 2 bảng
    └── greenhouse.db tệp cơ sở dữ liệu SQLite
```

Cấu trúc thư mục của dự án

3.3.2 Các chức năng xử lý chính

- **Kết nối MQTT:** Backend kết nối broker bằng `mqtt.connect`, subscribe `data_sensors` và `device_response`.
- **Nhận dữ liệu cảm biến:** khi có bản tin trên `data_sensors`, Backend ghi thêm thời gian và INSERT vào `sensor_data`.
- **Điều khiển thiết bị có xác nhận:** API điều khiển publish lệnh rồi tạo một `Promise` chờ phản hồi. Nếu bản tin `device_response` đến trước, Backend ghi kết quả thiết bị báo (SUCCESS/FAILED); nếu quá 5 giây, bộ đếm thời gian ghi FAILED và trả thông báo timeout.
- **Xác thực:** middleware `verifyJWT` đọc header `Authorization: Bearer <token>` và kiểm tra chữ ký bằng khóa bí mật, đồng thời đối chiếu với phiên đăng nhập đang lưu trong biến phienDangNhap (Backend chỉ giữ một phiên; đăng nhập mới hoặc đăng xuất làm token cũ mất hiệu lực); trả 401 nếu thiếu token, 403 nếu token sai, hết hạn hoặc đã đăng xuất.

```
app.post('/api/device/control', verifyJWT, (req, res) => {
  const { device, action } = req.body;
  if (!device || !action)
    return res.status(400).json({ error: "Thiếu thông tin device hoặc action!" });

  const payload = JSON.stringify({ device_id: "ESP8266-01", device, action });
  client.publish(TOPIC_CONTROL, payload);

  const result = new Promise((resolve) => {
    const timeoutTimer = setTimeout(() => {
      /* quá 5 giây: INSERT action_history với status = "FAILED"
         rồi resolve({ device, action, status: "FAILED", ... }) */
    }, 5000);
    pendingResponse = { resolve, timeoutTimer };  // giải phóng khi có
device_response
  });
  result.then(r => res.json(r));
});
```

Trích đoạn Backend (rút gọn): API điều khiển và cơ chế chờ phản hồi 5 giây

3.4 Danh mục API

Backend cung cấp 7 API theo kiểu REST, dữ liệu trao đổi ở dạng JSON. Trừ ba API xác thực (đăng nhập, đăng xuất, xem phiên đăng nhập), mọi API đều yêu cầu JWT trong header `Authorization: Bearer <token>`.

Bảng 11. Danh mục API của hệ thống

STT	Phương thức	Endpoint	Yêu cầu JWT	Chức năng
1	POST	/api/auth/login	Không	Đăng nhập, nhận JWT (hạn 2 giờ)
2	POST	/api/auth/logout	Không	Đăng xuất, hủy phiên đăng nhập ở Backend
3	GET	/api/login	Không	Xem tài khoản và token của phiên đang đăng nhập trên web (dùng kiểm thử bằng Postman)
4	GET	/api/dashboard	Có	Dữ liệu tổng hợp: số liệu mới nhất, dữ liệu biểu đồ, trạng thái 3 đèn (tham số limit)
5	POST	/api/device/control	Có	Gửi lệnh bật/tắt đèn qua MQTT và chờ phản hồi
6	GET	/api/sensors	Có	Toàn bộ dữ liệu cảm biến, mới nhất trước
7	GET	/api/history	Có	Toàn bộ lịch sử điều khiển, mới nhất trước

Các ví dụ dưới đây theo đúng cấu trúc dữ liệu mà mã nguồn trả về; giá trị minh họa lấy từ dữ liệu thực trong cơ sở dữ liệu.

a) API đăng nhập, đăng xuất

- **Phương thức, đường dẫn:** POST /api/auth/login
- **Dữ liệu gửi lên:** JSON gồm `username` và `password`.
- **Phản hồi:** HTTP 200 kèm `token` khi đúng; HTTP 401 khi sai thông tin.

```
POST /api/auth/login   {"username":"quantruong","password":"******"}
=> 200 {"success":true,"token":"eyJhbGciOiJIUzI1NiIs...",
        "message":"Đăng nhập thành công!"}
=> 401 {"success":false,"error":"Sai tài khoản hoặc mật khẩu!"}
```

Ví dụ gọi API đăng nhập

Đăng xuất: POST /api/auth/logout (không cần dữ liệu gửi lên) xóa phiên đăng nhập ở Backend và trả {"success":true,"message":"Đã đăng xuất!"}; token cũ bị từ chối với HTTP 403. Xem phiên đăng nhập: GET /api/login dùng khi kiểm thử bằng Postman, trả

username, password và token của phiên đang đăng nhập trên web; trả HTTP 401 nếu chưa có ai đăng nhập.

b) API điều khiển thiết bị

- **Phương thức, đường dẫn:** POST /api/device/control
- **Dữ liệu gửi lên:** `device` (LED01, LED02, LED03 hoặc ALL_LED) và `action` (ON hoặc OFF).
- **Phản hồi:** HTTP 200 kèm kết quả `status` là SUCCESS hoặc FAILED; HTTP 400 nếu thiếu tham số; 401/403 nếu token không hợp lệ.

```
POST /api/device/control   {"device":"LED01","action":"ON"}
=> 200 {"device":"LED01","action":"ON","status":"SUCCESS",
        "time":"2026/10/28 - 18:47:43"}


(khi ESP8266 không phản hồi sau 5 giây)
=> 200 {"device":"LED01","action":"ON","status":"FAILED","time":"...",
        "message":"Lỗi: Không kết nối được với phần cứng (Timeout sau 5 giây)!"}
```

Ví dụ gọi API điều khiển thiết bị

c) API dữ liệu cảm biến và lịch sử

- **Phương thức, đường dẫn:** GET /api/sensors và GET /api/history
- **Phản hồi:** HTTP 200, mảng JSON sắp xếp theo `id` giảm dần (bản ghi mới nhất đứng đầu); HTTP 500 nếu lỗi truy vấn.

```
GET /api/sensors
=> [ {"id":1413,"device_id":"ESP8266-01","temperature":26.5,
       "humidity":56.5,"light":723,"time":"2026/10/28 - 18:47:57"},
      {"id":1412, ... "temperature":26.5,"humidity":56.6,"light":701, ...},
      ... ]


GET /api/history
=> [ {"id":382,"device_id":"ESP8266-01","device":"LED03","action":"OFF",
       "status":"SUCCESS","time":"2026/10/28 - 18:47:52"}, ... ]
```

Ví dụ phản hồi API dữ liệu cảm biến và lịch sử (rút gọn)

d) API dashboard

GET /api/dashboard?limit=14 trả về một đối tượng gồm `latestSensor` (bản ghi cảm biến mới nhất), `chartData` (các bản ghi gần nhất theo thứ tự thời gian) và `deviceState` (trạng thái bật/tắt của LED01, LED02, LED03 tính từ lịch sử SUCCESS gần nhất). Giao diện hiện tại lấy dữ liệu từ `/api/sensors` và `/api/history` nên API này được giữ để phục vụ mở rộng.

3.5 Thiết kế giao diện Frontend

3.5.1 Cơ chế hoạt động chung

- **Một trang, nhiều màn hình:** thanh điều hướng bên trái chuyển giữa 4 trang bằng cách ẩn/hiện các thẻ `section`, không tải lại trang.

- **Xác thực:** khi mở trang, nếu chưa có JWT trong localStorage thì giao diện hiện form đăng nhập; sau khi đăng nhập thành công, `script.js` lưu JWT vào `localStorage` và đính kèm vào mọi yêu cầu. Nút "Đăng xuất" ở thanh bên trái, hoặc khi gặp 401/403, sẽ xóa token và quay về form đăng nhập.
- **Cập nhật dữ liệu:** cứ 2 giây giao diện gọi lại `/api/sensors` và `/api/history` rồi vẽ lại số liệu, biểu đồ và bảng (polling).
- **Trạng thái đèn:** được suy ra từ bản ghi SUCCESS gần nhất của từng đèn trong lịch sử, nên vẫn đúng sau khi tải lại trang.

3.5.2 Trang Dashboard

Trang Dashboard gồm ba khu vực: (1) ba thẻ số liệu nhiệt độ, độ ẩm, ánh sáng mới nhất; (2) biểu đồ đường vẽ bằng Canvas cho 3 đại lượng, trục trái 0-100 (°C và %), trục phải 0-1000 (ADC), người dùng chọn số bản ghi hiển thị là 7, 14, 30 hoặc 36 và rê chuột để xem chi tiết từng thời điểm; (3) khu vực điều khiển 3 đèn cùng hai nút "Bật tất cả" và "Tắt tất cả".

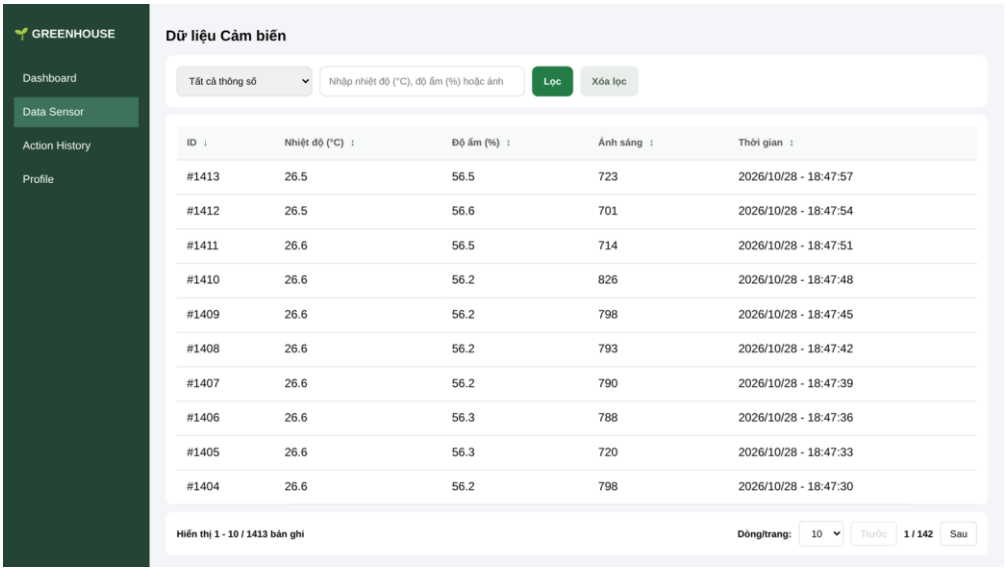
Trong lúc chờ thiết bị phản hồi, các nút bị khóa và dòng đèn hiển thị trạng thái "ĐANG XỬ LÝ" màu cam (tối thiểu 1,5 giây để người dùng kịp quan sát). Nút "Bật tất cả" mờ đi khi cả 3 đèn đã bật, nút "Tắt tất cả" mờ đi khi cả 3 đèn đã tắt.



Hình 9. Giao diện trang Dashboard

3.5.3 Trang Data Sensor

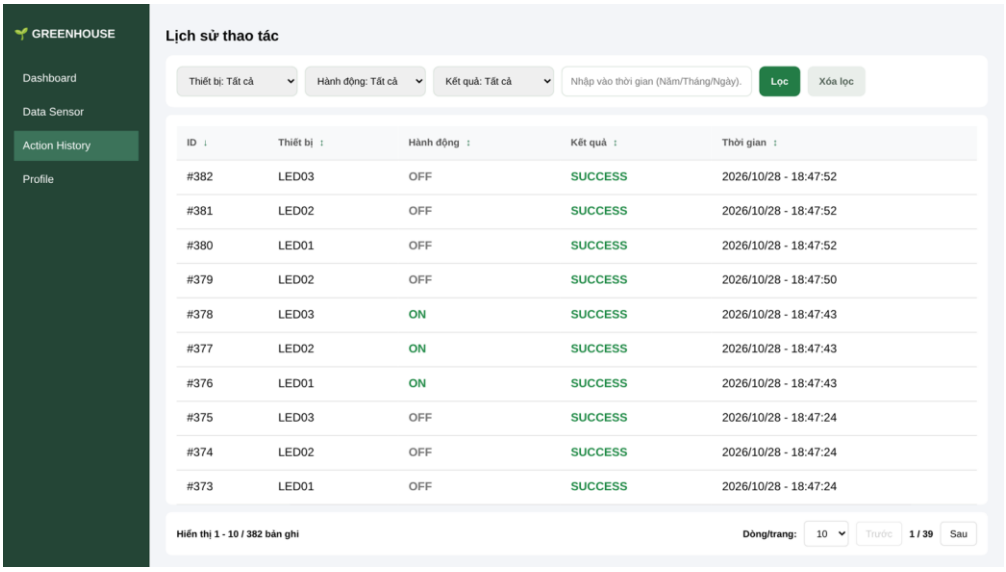
Trang hiển thị toàn bộ dữ liệu cảm biến dạng bảng gồm các cột ID, Nhiệt độ, Độ ẩm, Ánh sáng, Thời gian. Người dùng có thể tìm kiếm theo giá trị chính xác của một thông số hoặc theo ngày (định dạng Năm/Tháng/Ngày), sắp xếp tăng/giảm theo từng cột và phân trang với 5, 10 hoặc 20 dòng mỗi trang.



Hình 10. Giao diện trang Data Sensor

3.5.4 Trang Action History

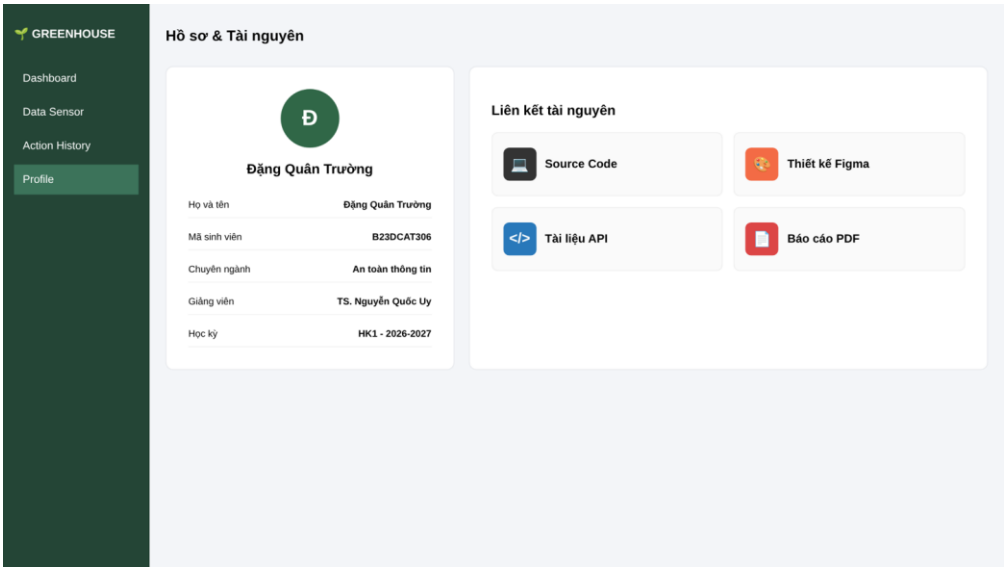
Trang hiển thị lịch sử điều khiển gồm các cột ID, Thiết bị, Hành động, Kết quả, Thời gian. Bộ lọc kết hợp được 4 điều kiện: thiết bị, hành động, kết quả và ngày; hành động ON và kết quả SUCCESS hiển thị màu xanh, kết quả FAILED màu đỏ để dễ nhận biết.



Hình 11. Giao diện trang Action History

3.5.5 Trang Profile

Trang Profile hiển thị thông tin sinh viên thực hiện (họ tên, mã sinh viên, chuyên ngành, giảng viên, học kỳ) và khu vực liên kết tài nguyên dự án gồm Source Code, thiết kế Figma, tài liệu API và báo cáo PDF. Mỗi mục là một liên kết: Source Code mở kho mã nguồn GitHub (https://github.com/quantruong998/IOT.git) và Thiết kế Figma mở bản thiết kế (https://www.figma.com/design/WUtFB9lG48OuZQ0QXlK1oz) trong tab trình duyệt mới; Tài liệu API và Báo cáo PDF tự động tải về tệp PDF tương ứng trong thư mục docs.

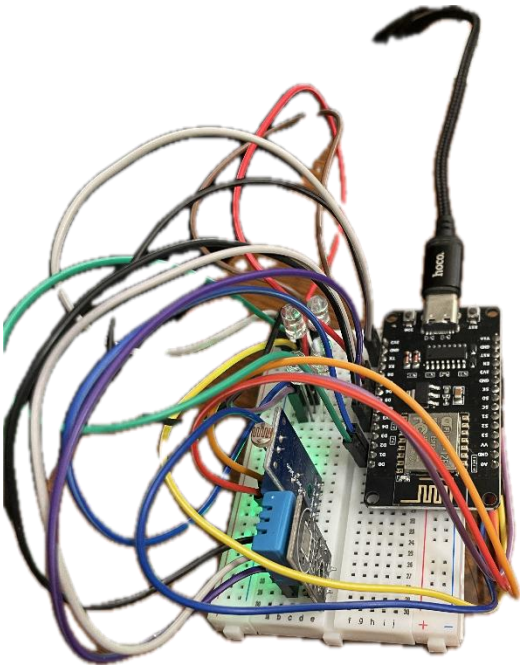


Hình 12. Giao diện trang Profile

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM

4.1 Mô hình phần cứng thực tế

Mạch được lắp trên breadboard theo sơ đồ chân ở mục 3.1: ESP8266 NodeMCU cấp nguồn và nạp chương trình qua cáp USB, DHT11 và quang trở đưa tín hiệu về D5 và A0, các đèn LED nối vào D1, D2, D6, tất cả dùng chung nguồn 3V3 và GND.



Hình 13. Mô hình phần cứng thực tế lắp trên breadboard

4.2 Số liệu vận hành thực tế

Số liệu dưới đây được thống kê trực tiếp từ tệp `greenhouse.db` tại thời điểm hoàn thiện báo cáo (28/10/2026).

Bảng 12. Thống kê dữ liệu trong cơ sở dữ liệu

Chỉ tiêu	Giá trị
Số bản ghi cảm biến (sensor_data)	1.728 bản ghi, từ 23/10/2026 17:55:51 đến 28/10/2026 22:50:52 (3 ngày đo: 23/10 có 238, 24/10 có 1.159, 28/10 có 331 bản ghi)
Chu kỳ gửi dữ liệu	3 giây (ví dụ các bản ghi #1409 đến #1413 cách nhau đúng 3 giây)
Nhiệt độ	Nhỏ nhất 26,5 °C; lớn nhất 31,2 °C; trung bình 29,7 °C
Độ ẩm	Nhỏ nhất 41,3 %; lớn nhất 68,6 %; trung bình 52,1 %
Ánh sáng	Nhỏ nhất 72; lớn nhất 959; trung bình 299 (đơn vị ADC)
Số lệnh điều khiển LED01-LED03 (action_history)	90 bản ghi, từ 23/10/2026 17:24:16 đến 28/10/2026 22:50:09: 72 SUCCESS và 18 FAILED
Thiết bị phần cứng	1 thiết bị: ESP8266-01

Bảng 13. Thống kê lệnh điều khiển theo từng đèn

Thiết bị	Bật thành công	Bật lỗi	Tắt thành công	Tắt lỗi	Tổng
LED01	12	5	9	4	30
LED02	10	2	14	0	26
LED03	14	3	13	4	34
Tổng	36	10	36	8	90

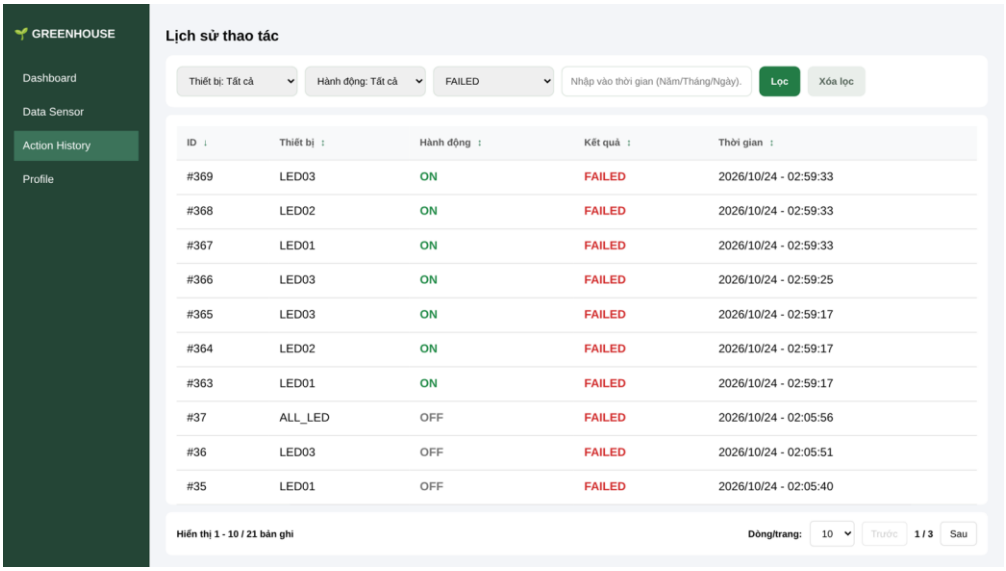
Ghi chú: ngoài 90 bản ghi trên, tệp cơ sở dữ liệu còn lưu các bản ghi từ những lần thử nghiệm ở phiên bản trước (lệnh ALL_LED ghi nguyên dạng và đèn cảnh báo LED_ALERT), không thuộc phạm vi của phiên bản cuối nên không đưa vào thống kê.

4.3 Kịch bản kiểm thử

Các kịch bản dưới đây kiểm chứng sự phối hợp giữa giao diện, Backend, MQTT, thiết bị và cơ sở dữ liệu. Bằng chứng là các bản ghi thực tế trong `action_history` và `sensor_data` sinh ra khi vận hành mạch thật.

Bảng 14. Các kịch bản kiểm thử và kết quả

STT	Kịch bản và thao tác	Kết quả	Bằng chứng
1	Bật một đèn. Gửi lệnh bật LED01 rồi bật LED03 (từ Dashboard).	Lệnh qua MQTT tới ESP8266; thiết bị phản hồi SUCCESS; đèn sáng; lịch sử ghi 1 dòng cho mỗi lệnh.	Bản ghi #343 (LED01 ON, 02:56:02) và #344 (LED03 ON, 02:56:04), ngày 24/10/2026
2	Bật/tắt tất cả. Gửi lệnh ALL_LED ON rồi ALL_LED OFF.	Cả 3 đèn đổi trạng thái cùng lúc; lịch sử có 3 dòng LED01, LED02, LED03 cho mỗi lệnh.	Bản ghi #370-#372 (ON, 18:47:22) và #373-#375 (OFF, 18:47:24), ngày 28/10/2026
3	Lệnh không thực hiện được. Gửi lệnh khi thiết bị mất kết nối hoặc đèn đã ở trạng thái yêu cầu.	Backend ghi FAILED; trạng thái đèn trên giao diện không đổi.	Bản ghi #363-#369 (FAILED, 24/10/2026); Hình 14
4	Chu kỳ cập nhật cảm biến. Quan sát thời gian các bản ghi liên tiếp.	Mỗi 3 giây có một bản ghi mới; Dashboard cập nhật theo.	Bản ghi #1409-#1413: 18:47:45, :48, :51, :54, :57
5	Tra cứu và lọc. Trang Action History, chọn kết quả "FAILED".	Bảng chỉ còn các lệnh FAILED, có phân trang (21 bản ghi, 3 trang).	Hình 14



Hình 14. Kết quả lọc lịch sử theo kết quả FAILED

Kết quả cho thấy chuỗi điều khiển hoạt động đúng thiết kế: lệnh chỉ được coi là thành công khi thiết bị phản hồi, lệnh thất bại được lưu lại thay vì bị bỏ qua, và dữ liệu cảm biến được cập nhật đều đặn theo chu kỳ 3 giây.

4.4 Đối chiếu kết quả với mục tiêu

Bảng 15. Đối chiếu kết quả thực nghiệm với mục tiêu đề ra

STT	Mục tiêu	Kết quả đạt được	Mức độ
1	Thu thập dữ liệu môi trường	ESP8266 đọc DHT11 và quang trở, gửi JSON mỗi 3 giây; CSDL lưu 1.728 bản ghi	Hoàn thành
2	Điều khiển có xác nhận	Bật/tắt đơn lẻ và tất cả; kết quả theo phản hồi thiết bị; timeout 5 giây	Hoàn thành
3	Lưu trữ dữ liệu	SQLite 2 bảng sensor_data, action_history	Hoàn thành
4	Backend và API	7 API REST, xác thực JWT, form đăng nhập/đăng xuất	Hoàn thành
5	Giao diện web	4 trang, biểu đồ, tìm kiếm, lọc, sắp xếp, phân trang, tự cập nhật	Hoàn thành
6	Bảo mật cơ bản	JWT cho API, MQTT có user/password; tài khoản cố định, chưa mã hóa đường truyền	Một phần
7	Điều khiển tự động theo ánh sáng (dự kiến ở Báo cáo 1)	Chưa hiện thực, đưa vào hướng phát triển	Chưa thực hiện

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Các kết quả đã đạt được

- **Phần cứng, firmware và truyền thông:** mạch ESP8266 đọc DHT11 và quang trở, điều khiển 3 đèn LED qua 3 topic MQTT tách biệt dữ liệu, lệnh và phản hồi; có phản hồi SUCCESS/FAILED cho từng lệnh; tự kết nối lại MQTT khi mất kết nối.
- **Backend:** REST API 7 endpoint, JWT, đăng nhập/đăng xuất, cơ chế chờ phản hồi có timeout 5 giây, ghi lịch sử đầy đủ kể cả khi thất bại.
- **Cơ sở dữ liệu:** SQLite 2 bảng, đã lưu 1.728 bản ghi cảm biến và 90 lệnh điều khiển trong quá trình vận hành.
- **Giao diện: form đăng nhập/đăng xuất và 4 trang web tự cập nhật,** có biểu đồ, tìm kiếm, lọc, sắp xếp, phân trang, điều khiển trực tiếp 3 đèn.

5.2 Những điểm hạn chế

- **Xác thực người dùng chưa hoàn chỉnh:** tài khoản đăng nhập và khóa bí mật JWT được ghi cố định trong mã nguồn, không có bảng người dùng và không có chức năng đăng ký. Backend chỉ giữ một phiên đăng nhập duy nhất (đăng nhập ở nơi khác làm phiên cũ mất hiệu lực), mật khẩu được giữ dạng văn bản thô trong phiên. API GET /api/login (phục vụ kiểm thử bằng Postman) trả cả tài khoản, mật khẩu và token mà không cần xác thực; cần gỡ bỏ khi triển khai thật.
- **Bảo mật đường truyền:** MQTT chạy qua TCP thường (không TLS), web chạy HTTP; thông tin đăng nhập MQTT nằm trong mã firmware.
- **Cập nhật bằng polling:** mỗi 2 giây giao diện tải lại toàn bộ `/api/sensors` và `/api/history`; lượng dữ liệu tăng dần theo thời gian nên sẽ chậm khi có nhiều bản ghi.
- **Chỉ xử lý một lệnh tại một thời điểm:** Backend dùng một biến `pendingResponse` dùng chung; giao diện khóa nút trong lúc chờ để tránh gửi chồng lệnh, nhưng gọi API trực tiếp liên tiếp có thể làm lệnh trước bị ghi đè. Backend cũng chưa kiểm tra giá trị hợp lệ của `device` và `action`.
- **Trạng thái FAILED gộp hai nguyên nhân:** thiết bị không phản hồi (timeout) và lệnh trùng trạng thái hiện tại đều ghi là FAILED, chưa phân biệt được lý do.
- **Firmware và cảm biến:** nhận lệnh bằng cách tìm chuỗi con trong bản tin (không dùng thư viện JSON); DHT11 độ chính xác thấp; ánh sáng là giá trị ADC, chưa quy đổi sang lux.
- **Chưa có chế độ tự động theo ngưỡng cảm biến** như dự kiến ở Báo cáo 1.

5.3 Hướng phát triển

- Bổ sung bảng người dùng, chức năng đăng ký, hỗ trợ nhiều phiên đăng nhập, mật khẩu băm (bcrypt), gỡ API GET /api/login và đưa khóa bí mật, mật khẩu MQTT ra biến môi trường.
- Dùng MQTT over TLS và HTTPS; thay polling bằng WebSocket hoặc Server-Sent Events; phân trang, lọc và sắp xếp ngay trên máy chủ.
- Gắn mã yêu cầu cho từng lệnh để xử lý nhiều lệnh song song và phân biệt lý do thất bại (timeout hay đã đúng trạng thái).
- Thêm chế độ tự động bật/tắt theo ngưỡng ánh sáng, nhiệt độ và cảnh báo khi vượt ngưỡng; mở rộng sang relay quạt, máy bơm thật; dọn dẹp dữ liệu cũ định kỳ.

TÀI LIỆU THAM KHẢO

[1] OASIS, *MQTT Version 3.1.1 Specification*, https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html

[2] Nick O'Leary, *PubSubClient - Arduino Client for MQTT*, https://pubsubclient.knolleary.net

[3] ESP8266 Arduino Core Documentation, https://arduino-esp8266.readthedocs.io

[4] Adafruit, *DHT sensor library*, https://github.com/adafruit/DHT-sensor-library

[5] Express.js Documentation, https://expressjs.com

[6] MQTT.js, *MQTT client for Node.js*, https://github.com/mqttjs/MQTT.js

[7] Auth0, *jsonwebtoken*, https://github.com/auth0/node-jsonwebtoken

[8] SQLite Documentation, https://www.sqlite.org/docs.html